

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

"C++ BUILD / IF / LOOP, PYTHON"

Реквизиты

Дмитрий Малов, Z33433, 2025 год

Цель работы

Познакомить студента с принципами компиляции исходного кода. Составить программу с использованием циклов, условий и функций. Сравнить быстродействие между C++ и Python. Ознакомление с типами данных.

Задача 1

[C++ EXPRESSION] Создать и скомпилировать программу на C++

Результат сборки (компиляции) сохранять в папку build. Папку build сделать игнорируемой для GIT. Программа должна получать на вход число – это количество итераций для выполнения расчета. В рамках итерации выполнять следующее вычисление: $x^2 - x^2 + x \cdot 4 - x \cdot 5 + x + x$. Вычисление выполнять в виде отдельной от main функции, которая будет вызвана циклически из main. Фиксировать время выполнения программы, затрачиваемое на расчет выражения n раз (n задается в консоли перед вычислением). Предусмотреть дополнительный цикл на повторную итерацию запуска программы вычислений. Если было введено не число, то завершить выполнение программы.

Задача 2

[PYTHON EXPRESSION] Создать и скомпилировать программу на Python 3

Результат сборки (компиляции) сохранять в папку build. Папку build сделать игнорируемой для GIT. Программа должна получать на вход число – это количество итераций для выполнения расчета. В рамках итерации выполнять следующее вычисление: $x^2 - x^2 + x \cdot 4 - x \cdot 5 + x + x$. Вычисление выполнять в виде отдельной от main функции, которая будет вызвана циклически из main. Фиксировать время выполнения программы, затрачиваемое на расчет выражения n раз (n задается в консоли перед вычислением). Предусмотреть дополнительный цикл на повторную итерацию запуска программы вычислений. Если было введено не число, то завершить выполнение программы.

Решение задач 1 и 2

- Для решения задач 1 и 2 на языках Python и C++ реализуем набор функций, каждая из которых будет выполнять аналогичные действия в разных языках программирования.
- Реализуем функцию iteration(x). Эта функция будет получать на вход число x и выполнять вычисление, предложенное в условии.

```
def iteration(x):  
    return x ** 2 - x ** 2 + x * 4 - x * 5 + x + x
```

```
double iteration(double x) {
    return x*x - x*x + x*4 - x*5 + x + x;
}
```

- Реализуем функцию run_experiment(n, x). Данная функция будет вычислять n итераций предложенного выражения. Также эта функция будет замерять время, потраченное на расчёт и выводить его.

```
def run_experiment(n, x):
    start_time = time.time()
    for _ in range(n):
        iteration(x)

    sum_time = time.time() - start_time
    print(f"Время {n} итераций: {sum_time:.6f}")
```

```
void run_experiment(int n, int x) {
    auto start_time = std::chrono::high_resolution_clock::now();

    for (int i = 0; i < n; ++i) {
        iteration(x);
    }

    auto end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> sum_time = end_time - start_time;

    std::cout << "Time of " << n << "iterations: " << sum_time.count() << std::endl;
}
```

- Реализуем функцию is_number(s). Эта функция будет получать на вход значение, введенное пользователем, и определять, является ли это значение числом (в дальнейшем эта функция понадобится, поскольку мы хотим организовать работу программы так, что вызов из программы производится посредством ввода не числовой строки)

```
def is_number(s):
    try:
        int(s)
        return True
    except ValueError:
        return False
```

```
bool is_number(std::string s) {
    for (int i = 0; i < s.length(); ++i) {
        if (!isdigit(s[i])) {
            return false;
        }
    }
    return true;
}
```

- Наконец, реализуем основную функцию `main()`. Она будет циклически вызывать вычисления в заданном пользователем количестве, пока не будет введена не числовая строка.

```
def main():
    while True:
        user_input = input("Введите количество итераций (или не число для выхода из программы)")

        if not is_number(user_input):
            print("Работа программы завершена")
            break

        n = int(user_input)
        x = random.randint(1, 100000)
        run_experiment(n, x)
```

```
int main() {
    while (true) {
        std::cout << "Enter number of iterations (or non-number to exit): ";
        std::string input;
        std::cin >> input;

        if (!is_number(input)) {
            std::cout << "Program completed" << std::endl;
            break;
        }

        int n = std::stoi(input);

        int x = rand();
        run_experiment(n, x);
    }

    return 0;
}
```

- Остаётся только запустить функцию `main()`
- Результаты работы программ:

```
Enter number of iterations (or non-number to exit): 1
Time of 1iterations: 0
Enter number of iterations (or non-number to exit): 10
Time of 10iterations: 0
Enter number of iterations (or non-number to exit): 100
Time of 100iterations: 0
Enter number of iterations (or non-number to exit): 1000
Time of 1000iterations: 0
Enter number of iterations (or non-number to exit): 10000
Time of 10000iterations: 0
Enter number of iterations (or non-number to exit): 100000
Time of 100000iterations: 0.0009987
Enter number of iterations (or non-number to exit): 1000000
Time of 1000000iterations: 0.0100012
Enter number of iterations (or non-number to exit): 10000000
Time of 10000000iterations: 0.0909997
Enter number of iterations (or non-number to exit): 100000000
Time of 100000000iterations: 0.907999
Enter number of iterations (or non-number to exit): exit
Program completed
```

Рис. 1: C++

```
Введите количество итераций (или не число для выхода из программы)1
Время 1 итераций: 0.000000
Введите количество итераций (или не число для выхода из программы)10
Время 10 итераций: 0.000000
Введите количество итераций (или не число для выхода из программы)100
Время 100 итераций: 0.000000
Введите количество итераций (или не число для выхода из программы)1000
Время 1000 итераций: 0.000000
Введите количество итераций (или не число для выхода из программы)10000
Время 10000 итераций: 0.002997
Введите количество итераций (или не число для выхода из программы)100000
Время 100000 итераций: 0.020004
Введите количество итераций (или не число для выхода из программы)1000000
Время 1000000 итераций: 0.208997
Введите количество итераций (или не число для выхода из программы)10000000
Время 10000000 итераций: 2.097999
Введите количество итераций (или не число для выхода из программы)100000000
Время 100000000 итераций: 20.918523
Введите количество итераций (или не число для выхода из программы)exit
Работа программы завершена

Process finished with exit code 0
```

Рис. 2: Python

Заключение

По результатам работы программы видим, что на полностью аналогичные действия программа C++ потратила примерно в 20 раз меньше времени. Это связано с особенностями языков, а именно тем, Python не компилируется и не имеет жёсткой типизации.